



Instituto Tecnológico de Costa Rica

Compiladores e interpretes

Proyecto LogoTec
II Sprint

Profesor: Marco Hernández

Integrantes:
Andres Blanco 2022108841

Giancarlo Vega 2020195338

Sebastián Chaves 2021032506

II Semestre

2025

Para este avance, se optó por realizar la generación de código intermedio, la optimización, la compilación en ensamblador y la generación de código ejecutable por medio de clases Java, optando por no utilizar herramientas como LLVM o GCC. Se mantuvo la gramática hecha en ANTLR4, utilizada en el primer sprint.

Generación de Código Intermedio (IR)

La generación de código intermedio es una etapa importante del backend del compilador. Su objetivo es tomar el AST (Abstract Syntax Tree) producido en las fases anteriores y convertirlo en un formato intermedio que sea más cercano al código de máquina, pero independiente de la plataforma. Este formato permite realizar optimizaciones y simplifica la traducción posterior a ensamblador o a un archivo objeto ejecutable.

IRbuilder

Este recorre el AST y traduce cada nodo a instrucciones intermedias que representan operaciones aritméticas y lógicas, asignaciones, estructuras de control, llamadas a funciones y comandos del lenguaje, como los de tortuga. Al crearse, inicializa un objeto IR vacío que almacenará las instrucciones globales y funciones del programa. El método build recorre cada nodo y llama a translateStatement, que identifica el tipo de nodo y lo traduce. Las asignaciones generan instrucciones, mientras que las declaraciones de variables no. Las expresiones se descomponen en operaciones simples usando variables temporales y los condicionales y bucles se convierten en etiquetas y saltos que mantienen el flujo original.

Para las funciones, IRBuilder crea objetos Function con bloques de instrucciones y retorno final, y traduce las llamadas generando instrucciones para los parámetros y un Call. Los comandos de tortuga se convierten en llamadas específicas con sus argumentos. Los nombres de variables temporales y etiquetas se generan automáticamente, y se usa reflexión para acceder a los campos del AST sin modificar la traducción manualmente.

El resultado es un IR secuencial que refleja la lógica del programa original, listo para optimizaciones y para ser traducido a ensamblador o archivo objeto, funcionando como puente entre el AST y el código ejecutable.

Optimización de Código

Constant Folding

La clase ConstantFolder recorre el IR para detectar operaciones cuyos operandos son constantes y calcular sus resultados en tiempo de compilación, reduciendo así instrucciones innecesarias. Primero analiza las instrucciones globales y luego cada función, manteniendo un mapa de constantes conocidas para propagar valores. Asignaciones, operaciones binarias y unarias se evalúan si sus operandos son literales o ya conocidos, reemplazando la instrucción por un valor fijo y aumentando un contador de constantes plegadas.

Para determinar si un operando es constante, se verifica si es un número, un booleano o una variable previamente evaluada. Las operaciones binarias y unarias se calculan directamente y el resultado se guarda como constante para futuras instrucciones. El proceso asegura que el IR resultante tenga menos operaciones redundantes, manteniendo la lógica del programa original y preparando el código para fases posteriores de optimización o generación de ensamblador.

Peephole Optimization

La clase Peephole realiza optimizaciones locales en el IR, identificando patrones simples de operaciones que pueden simplificarse. Recorre las instrucciones globales y las de cada función, aplicando reglas que eliminan cálculos innecesarios, como sumar cero, multiplicar por uno o combinar valores booleanos con true o false. Cada vez que detecta un patrón aplicable, reemplaza la operación original por una instrucción más simple, reduciendo el número de pasos sin alterar el resultado del programa. El contador optimizedCount lleva registro de cuántas optimizaciones se realizaron.

Dead Code Elimination

La clase DeadCodeElim elimina código muerto del IR, es decir, instrucciones que calculan valores nunca utilizados. Para esto, primero hace un recorrido hacia atrás marcando todas las variables que se usan en operaciones, retornos o llamadas. Luego, en un recorrido hacia adelante, elimina cualquier asignación o cálculo cuyo resultado no sea usado y que provenga de variables temporales generadas automáticamente. El contador eliminatedCount registra cuántas instrucciones fueron eliminadas. Esta optimización reduce instrucciones

innecesarias, limpiando el IR y haciendo el código final más eficiente sin cambiar su comportamiento.

Generación de Código de Máquina (ASM)

SimpleAsmGenerator

Esta clase convierte el IR optimizado en instrucciones de ensamblador simples. Recorre primero las instrucciones globales y luego cada función del programa, generando etiquetas para bloques y nombres de función. Cada instrucción del IR se traduce en un comando ASM equivalente: asignaciones se vuelven MOV, operaciones binarias y unarias se vuelven BIN y UN con su operador correspondiente, saltos condicionales y no condicionales se traducen a JMPT, JMPTF y JMP, y las llamadas a funciones y retornos se convierten en CALL y RET. Los parámetros de función se manejan con PUSH y, si la llamada retorna un valor, se almacena con POP. Esto produce un listado secuencial de instrucciones que refleja fielmente la lógica del programa y permite su ejecución directa en una arquitectura de ensamblador simulada.

Análisis Semántico

SemanticAnalyzer

Este verifica que un programa cumpla las reglas del lenguaje antes de la ejecución. Mantiene una tabla de símbolos que registra las variables declaradas, su tipo, valor y si han sido inicializadas. Comprueba que la primera línea tenga un comentario y que exista al menos una variable declarada. También controla las funciones: registra las declaradas y las llamadas, generando errores si se llaman funciones no definidas.

Cuando se declara o asigna una variable, se valida el nombre y se asegura que el tipo sea compatible con el existente, actualizando su valor e inicialización. Para expresiones, SemanticAnalyzer puede inferir su tipo examinando constantes, referencias a variables o nodos AST, y verifica compatibilidad de tipos entre expresiones.

El analizador detecta errores semánticos y léxicos, los almacena y permite lanzar una excepción si se encuentran problemas. Además, ofrece funciones de depuración que

muestran la tabla de símbolos y el estado interno, ayudando a entender la estructura del programa y a rastrear problemas durante la compilación.

`SemanticAnalyzerVisitor`

Esta clase recorre el árbol generado por ANTLR para un programa y aplica el análisis semántico. Su función principal es conectar cada nodo del AST con las verificaciones de `SemanticAnalyzer`. Al iniciar, revisa si la primera línea del código contiene un comentario.

Cuando encuentra declaraciones de variables, asignaciones o inicializaciones, registra cada variable, infiere su tipo y verifica que las asignaciones sean compatibles con el tipo existente. Para las funciones, marca cuáles se declaran y cuáles se llaman, permitiendo detectar llamadas a funciones no definidas.

Cada nodo de expresión se visita para determinar su tipo y asegurar consistencia semántica. Los comentarios se ignoran después de la primera línea, ya que solo se requiere verificar el comentario inicial. Al finalizar el recorrido, el visitante llama al análisis final de `SemanticAnalyzer`, garantizando que se cumplan todas las reglas del lenguaje antes de la compilación o generación de código.

Ejecución del Programa Compilado

La clase `RuntimePlayer` es la encargada de ejecutar un programa ya compilado en formato `.lobj`. Su funcionamiento se puede describir en varios pasos conectados. Primero, el programa se carga desde un archivo de texto, leyendo línea por línea y almacenando todas las instrucciones en una lista interna. Durante este proceso, identifica las etiquetas de control y las funciones definidas, guardando sus posiciones para poder saltar a ellas durante la ejecución.

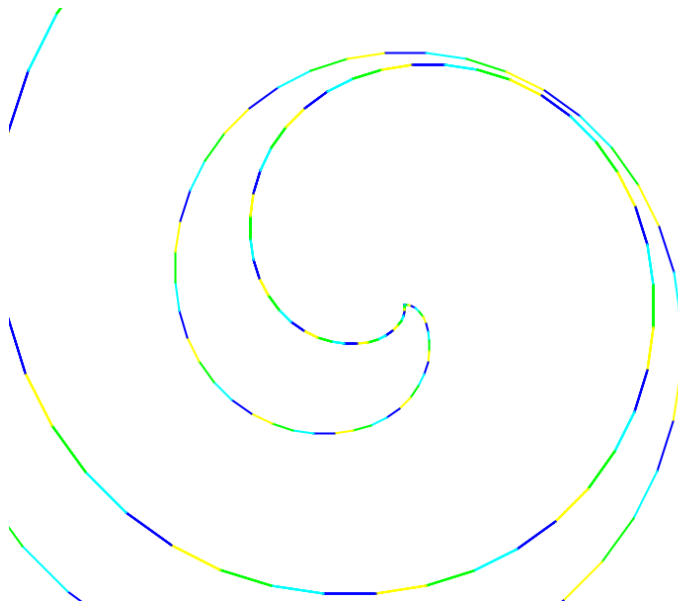
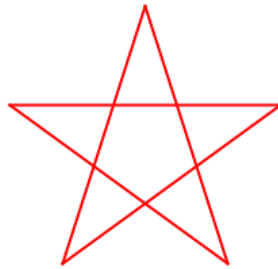
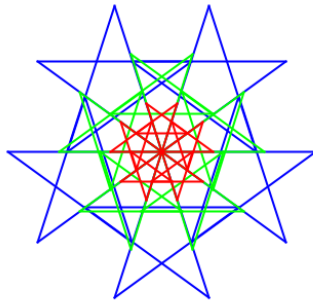
Una vez cargado el programa, la ejecución comienza con un recorrido secuencial de las instrucciones globales mediante un contador de programa (pc). Cada instrucción se interpreta según su tipo. Por ejemplo, las asignaciones (MOV) almacenan valores en un mapa de memoria, mientras que las operaciones binarias (BIN) y unarias (UN) calculan resultados usando los valores existentes en la memoria. Las instrucciones de control de flujo (JMP, JMPT, JMPTF) permiten saltar a etiquetas específicas según condiciones evaluadas.

Para el manejo de funciones, RuntimePlayer utiliza una pila. Los argumentos de las funciones se colocan en la pila mediante PUSH y se recuperan con POP. Cuando se llama a una función (CALL), se guarda el estado actual de la memoria y el contador de programa, se establece un nuevo entorno de variables locales y se saltan las instrucciones correspondientes a la función. Al retornar (RET), se restaura el contexto anterior y se devuelve el valor de retorno si existe.

El motor también soporta funciones integradas o built-in, como println para imprimir valores, random para generar números aleatorios, y un conjunto de comandos para controlar un objeto tortuga (TurtleRuntime), que permiten moverla, girarla, cambiar el color del lápiz o controlar su visibilidad. Los valores de las operaciones y variables se obtienen mediante un método que interpreta literales y referencias de memoria, asegurando que las operaciones aritméticas y lógicas se ejecuten correctamente.

Finalmente, si el programa genera instrucciones de dibujo con la tortuga, RuntimePlayer utiliza TurtleViewer para mostrar la imagen en pantalla y guardarla en un archivo con un nombre único basado en la fecha y hora. De esta manera, la clase actúa como una máquina virtual completa, capaz de manejar memoria, pila, flujo de control, funciones y salida gráfica de forma organizada y coherente.

Pruebas Realizadas



Problemas encontrados y solucionados

Ocurrieron problemas con la ejecución de las clases que manejaban el análisis semántico de errores. Obtenía datos incorrectos, o del todo no los leía bien. Daba falsos positivos y falsos negativos, así como aumento en el tiempo de ejecución del código. Se logró resolver modificando la gramática, y optimizando las clases relacionadas a esta tarea.

Se encontraron problemas en los códigos de Python que manejaban la creación de los archivos .IR, .ASM, y .lobj, causando malas compilaciones, y por consecuencia, malas ejecuciones. Entre los problemas específicos, era que en el Frontend funcionaban funciones como el repite, inc y hasta, pero en la traducción del backend ocurrían problemas. Se logró resolver al modificarlos basados en las reglas específicas de la gramática hecha en ANTLR4.

Se presentaron errores a la hora de crear los dibujos. Había fallas relacionadas al error previamente mencionados, y otras donde no se realizaba bien el dibujo acorde al código ejecutado. Se logró resolver esos errores, y los dibujos se generaban al ejecutar el código.

Repositorio

Link: https://github.com/jungianca6/gvega_compiladores_interpretes_2025

Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2.^a ed.). Pearson. <https://www.pearson.com/en-us/subject-catalog/p/compilers-principles-techniques-and-tools/P200000003472/9780133002140>

ANTLR Project. (n.d.). *ANTLR 4 documentation*. GitHub. Recuperado el 4 de octubre de 2025, de <https://github.com/antlr/antlr4/blob/master/doc/index.md>

Appel, A. W. (2002). *Modern compiler implementation in Java*. Cambridge University Press. <https://www.cambridge.org/core/books/modern-compiler-implementation-in-java/34EACED718B1D6D5237705F9BFD7CD4A>

Brown, S. (n.d.). *The C4 model for visualising software architecture*. Recuperado el 4 de octubre de 2025, de <https://c4model.com/>

Cooper, K. D., & Torczon, L. (2022). *Engineering a compiler* (3.^a ed.). Morgan Kaufmann/Elsevier. <https://shop.elsevier.com/books/engineering-a-compiler/cooper/978-0-12-815412-0>

ISO/IEC/IEEE. (2022). *ISO/IEC/IEEE 42010:2022 — Software, systems and enterprise — Architecture description*. International Organization for Standardization. <https://www.iso.org/standard/74393.html>

LLVM Project. (n.d.). *LLVM language reference manual*. Recuperado el 4 de octubre de 2025, de <https://llvm.org/docs/LangRef.html>

Nystrom, R. (2021). *Crafting interpreters*. Genever Benning. <https://craftinginterpreters.com/>

Parr, T. (2013). *The definitive ANTLR 4 reference*. Pragmatic Bookshelf. <https://pragprog.com/titles/tpantlr2/the-definitive-antlr-4-reference/>

PlantUML Team. (n.d.). *PlantUML language reference guide*. Recuperado el 4 de octubre de 2025, de https://pdf.plantuml.net/PlantUML_Language_Reference_Guide_en.pdf